

Reviewer Pack — Free Cumulant Spread Bounds Disjointness Communication Compl...

Entry 08de756b0357 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 04:06:28 UTC

Free Cumulant Spread Bounds Disjointness Communication Complexity

Entry ID: 08de756b0357 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 04:06:28 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For a DISJOINTNESS matrix $M \in \{0,1\}^{\{n \times n\}}$, let $\tau(M)$ denote the spread of free cumulants of its entries. Then $\tau(M) \geq \Omega(1/\sqrt{n})$ and $\tau(M) \leq O(\log n)$. The exact relation $\tau(M) = \Theta(1/\sqrt{n})$ holds for all $n \geq 2$.

Rationale (proposer's reasoning):

Free cumulants quantify non-commutative dependencies in random variables. DISJOINTNESS requires coordinated information exchange, which may manifest as structured dependencies detectable via free cumulant spreads. This bridges operator-algebraic structure to communication lower bounds.

Taxonomy category: COMM_DISJ (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 951d43d25d2e2db6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    n = 40
    random.seed(seed)

    # Generate a DISJOINTNESS matrix  $M \in \{0,1\}^{n \times n}$ 
    M = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    while not is_disjointness(M):
        M = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    # Compute the free cumulant spread  $\tau(M)$ 
    tau_M = compute_free_cumulant_spread(M)

    # Check if  $\tau(M) \geq 1/\sqrt{40}$  and  $\tau(M) \leq \log 40$ 
    lower_bound = 1 / math.sqrt(n)
    upper_bound = math.log(n)
    conjecture_holds = lower_bound <= tau_M <= upper_bound

    return {
        "metric_name": "free_cumulant_spread",
        "metric_value": tau_M,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

def is_disjointness(M):
    n = len(M)
    for i in range(n):
        for j in range(i + 1, n):
            if sum(M[i][k] * M[j][k] for k in range(n)) > 0:
                return False
    return True

def compute_free_cumulant_spread(M):
    n = len(M)
    R_transform = [[0.0] * n for _ in range(n)]
```

```

# Compute the R-transform
for i in range(n):
    for j in range(i, n):
        if i == j:
            R_transform[i][j] = M[i][i]
        else:
            R_transform[i][j] = (M[i][j] - 1) / (1 + M[i][i])

# Compute the free cumulant spread
tau_M = 0.0
for i in range(n):
    for j in range(i, n):
        if i == j:
            tau_M += R_transform[i][j]
        else:
            tau_M += abs(R_transform[i][j])

return tau_M

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing verification of support/falsification conditions. | next: Increase timeout duration and re-run test with extended computation limits

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	113376
2	propose	ollama_remote	qwen3:8b	0	0	56896
3	preregistration	ollama_remote	qwen3:8b	0	0	12956
4	novelty	ollama_remote	qwen3:8b	0	0	10610
5	novelty	ollama_remote	qwen3:8b	0	0	8168
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12183
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9327
8	judge	ollama_remote	qwen3:8b	0	0	17526

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 241040 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/08de756b0357.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/08de756b0357.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/08de756b0357.tar.gz` (if generated)